VINUNIVERSITY

Object Detection and Data

AICB-P2T4 · Ngày 02 · Detector hoạt động thế nào, xây từ con số 0, đo ra sao

Tên Giảng Viên

VinUniversity · Data Track

HÃY SUY NGHĨ...

“Chiếc xe nhìn thấy gì trước mặt nó
— và vì sao đôi khi nó không thấy?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học

1. Object Detection Và Xe Tự Lái
2. Bên Trong Một Detector
3. Đo Detector Và Đo Dữ Liệu
4. Dữ Liệu Dạy Detector
5. Họ YOLO Và Detection 2026
6. Xây Một Detector Từ Con Số 0
7. Thực Hành

Hôm nay bạn sẽ

VINUNIVERSITY

Hiểu

- Detector trả về gì, bên trong nó làm gì.
- IoU, precision, recall, mAP: đọc được, tính được.
- Detection 2026: model nào, làm được gì, sai ở đâu.

Làm

- Tự xây một detector nhỏ: thu ảnh, tạo dữ liệu, train.
- Chạy nó trên ảnh nó chưa từng thấy.
- Tìm chỗ nó sai — rồi quyết định sửa dữ liệu hay đổi ngưỡng.

Sáng: detector hoạt động thế nào, đo ra sao. Chiều: tự xây một detector và tìm chỗ nó sai.

Slide gắn ●●○ Nâng cao hoặc ●●● Chuyên sâu dành cho bạn đã biết — người mới có thể bỏ qua.

Object Detection Và Xe Tự Lái

01

Detector trả về gì, và những bộ phận nào phụ thuộc vào nó

Detection đang chạy quanh bạn

Điện thoại
Tìm khuôn mặt trong khung hình, ngay trên máy.

Tìm ra mặt — không nhận ra là ai.

Nội soi đại tràng
Vẽ box xanh quanh polyp ngay trong ca nội soi.

Thử nghiệm ngẫu nhiên: tỉ lệ ca tìm thấy ≥ 1 u tuyến 55,1% so với 42,0%.

Ô tô
Tự phanh khi thấy người đi bộ, xe đạp, xe máy.

Euro NCAP thử cả tình huống xe máy từ 2023.

Bẫy ảnh động vật
MegaDetector tìm động vật, người, xe trong ảnh bẫy.

Loại nhanh hàng loạt ảnh trống.

Cửa hàng không thu ngân
Camera trên trần nhận ra lúc tay khách chạm vào hàng.

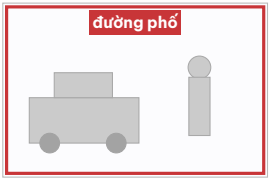
Tự tính tiền khi khách bước ra.

Nhà máy
Tìm lỗi trên tấm mạch, tấm wafer, rồi xếp loại lỗi.

NVIDIA 2025: xếp loại lỗi đúng 93,84% $\rightarrow$ 98,51% sau khi tinh chỉnh.

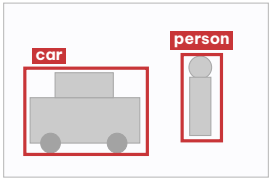
Lõi chung: **lớp** + **điểm** + **box**; mỗi hệ thống ghép thêm bước riêng. Khác nhau nhiều nhất ở **dữ liệu** dạy nó.

Cùng một ảnh: ba bài toán, ba câu trả lời



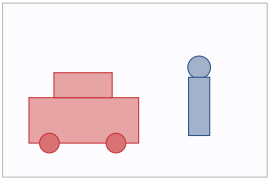
Classification

Ảnh có gì?
1 nhãn cho cả ảnh



Object detection

Có gì, ở đâu, bao nhiêu?
box + nhãn cho mỗi vật



Segmentation

Pixel nào thuộc vật?
nhãn cho từng pixel

Hôm nay: **detection**. Segmentation (Ngày 05) chi tiết hơn nhưng tốn nhiều công gán nhãn hơn. Box cho vị trí — khoảng cách lấy từ LiDAR, radar hoặc camera đã hiệu chỉnh.

Detector trả về cái gì

Detection

class + confidence + bounding box

```
pedestrian, 0.87, (412, 236, 68, 154)
```

bốn số ở đây là (x, y, w, h): góc trên-trái, rộng, cao — tính bằng pixel

Ảnh vào, **danh sách dự đoán** ra — model vẫn có thể bỏ sót vật. Dữ liệu dùng để train có **cùng hai thứ** — lớp và box — chỉ thiếu confidence: đó là đáp án model được chấm theo. Bốn số của box thì mỗi định dạng ghi một kiểu (sắp gặp).

Giữ hay bỏ dòng thứ ba?

```
person   0.91   (x1=412, y1=118, x2=530, y2=447)
car      0.88   (x1=  8, y1=302, x2=291, y2=470)
person   0.34   (x1=901, y1=287, x2=944, y2=372)   <- giu hay bo?
```

Giơ tay trước

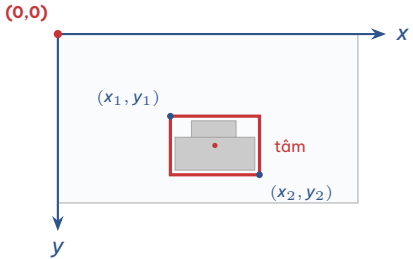
Giữ dòng thứ ba hay bỏ? Mỗi phía nói **một lý do**.

Rồi mới nói

0,34 không có nghĩa "34% là người". Đó là điểm nội bộ; hai model cùng in 0,9 có thể đúng với tỉ lệ khác nhau.

Lưu ý: Ngưỡng giữ/bỏ là **một quyết định theo mục đích**. Xe đưa đón học sinh và máy đếm xe qua ngã tư sẽ chọn khác nhau — không có con số "đúng" đọc ra từ điểm.

Bốn con số của một box



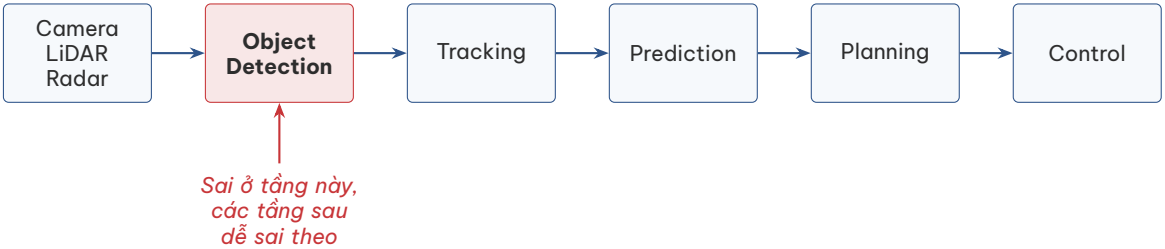
Gốc ở **góc trên-trái**, trục *y* hướng **xuống**

Cách viết	Gặp ở đâu
x_1, y_1, x_2, y_2	hai góc, pixel — kết quả model in ra
x, y, w, h	góc trên-trái + rộng, cao, pixel — file COCO
c_x, c_y, w, h	**tâm** + cỡ, chia cho cỡ ảnh (0–1) — file nhãn YOLO

Đổi một dòng YOLO ra pixel (ảnh 1920×1080)

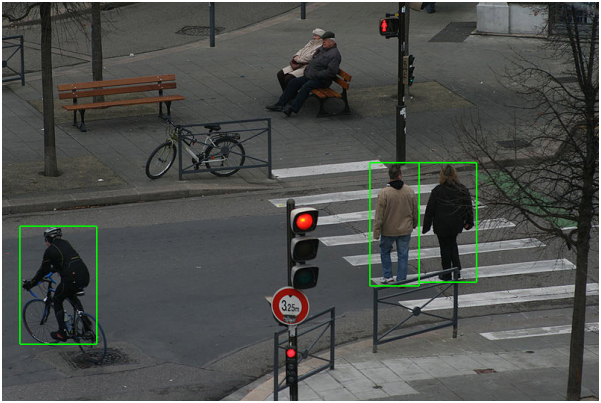
`2 0.5 0.5 0.1 0.2` $\Rightarrow$ tâm $(960, 540)$, cỡ 192×216 $\Rightarrow$
x_1=864, y_1=432, x_2=1056, y_2=648. Đọc nhầm tâm thành góc thì **mọi box lệch nửa kích thước**.

Detection nằm ở đâu trong xe tự lái



Trong chuỗi mô-đun này, mỗi tầng chỉ thấy đầu ra của tầng trước. Detection bỏ sót một người đi bộ $\Rightarrow$ **các tầng sau rất khó biết người đó tồn tại**.

Detector nhìn thấy gì — và bỏ sót gì



Ba box xanh: model đã tìm ra một người đi xe đạp và hai người đi bộ.

Nhưng hãy nhìn kỹ hơn:

- hai người ngồi trên ghế — **không có box**
- chiếc xe đạp dựng bên trái — **không có box**

Nếu vật thuộc lớp cần tìm, thiếu box dự đoán là **false negative**. Thiếu box trong đáp án train là một lỗi dữ liệu khác.

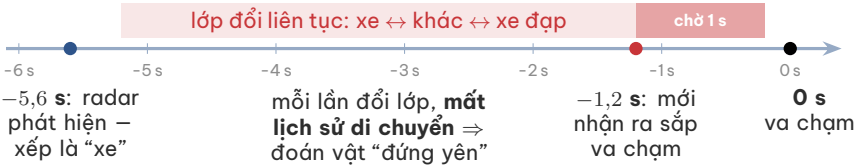
Nguồn: Wikimedia Commons — CC BY-SA 2.0

Hai kiểu lỗi, hai hậu quả

- **False negative** — có vật thể, model không thấy.
  Xe *có thể không phanh* trước người đi bộ. **Nguy hiểm**.
- **False positive** — không có vật thể, model báo có.
  Xe *có thể phanh gấp* vì một cái bóng. Mất tin cậy.

Lưu ý: Một người đi bộ **bị bỏ sót trong dữ liệu train** dạy model rằng "ở đây không có người đi bộ". Đó là dữ liệu *sai*, không chỉ là dữ liệu thiếu.

Tempe 2018: thấy từ sớm, hiểu quá muộn



Hệ thống **chưa lần nào** xếp người này vào lớp người đi bộ. Khi nhận ra nguy hiểm, thiết kế cho nó **chờ 1 giây, không phanh, không cảnh báo** — vì lo báo nhầm.

Bài học cho detection

Đúng ở từng khung hình chưa đủ: lớp phải **ổn định theo thời gian** (Ngày 3). Sợ báo nhầm (FP) cũng có giá: **bỏ sót** (FN).

Nguồn: NTSB, báo cáo HAR-19/03. Theo NTSB, nguyên nhân chính là người vận hành trên xe mất tập trung; thiết kế hệ thống là một trong các yếu tố góp phần.

Dữ liệu từ đâu, model giỏi ở đó

VINUNIVERSITY

Xe máy: hiếm hay phổ biến?

BDD100K (đường Mỹ): xe máy chỉ khoảng **0,2%** số vật thể có box.

DriveIndia (đường Ấn Độ, 2025): xe máy là **một trong ba lớp nhiều nhất**; model YOLO đạt mAP50 0,947 với lớp này.

Ban ngày hay ban đêm?

Train chỉ bằng ảnh **ban ngày**, thử trên ảnh **ban đêm**: AP (average precision, chương 3) tụt từ **30,6** xuống **23,6** (BDD100K). Khoảng chênh ngày–đêm lớn hơn nhiều so với thành phố–ngoài thành phố.

Đông Nam Á là đường của xe máy

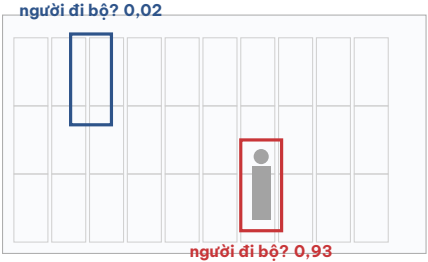
ASEAN NCAP: người đi xe máy chiếm **80%** người tham gia giao thông trong khu vực. Quy trình 05/2024 chấm điểm riêng việc xe **tự phanh khi phát hiện xe máy**. Model giỏi trên dữ liệu nước ngoài chưa chắc giỏi ở ngã tư của mình.

Bên Trong Một Detector

02

Từ lưới pixel đến vài chiếc box: ứng viên, ngưỡng, NMS

Ý tưởng đầu tiên: trượt khung khắp ảnh



Ảnh 1920×1080, khung 64×128, mỗi bước 8 px:

233×120 = **27.960 khung** — cho *một* cỡ khung.

Người ở gần thì to, ở xa thì nhỏ: cần khoảng 10 cỡ khung

$\Rightarrow$ **gần 280.000 lần** chạy bộ phân loại *cho mỗi ảnh*.

Hai cách bớt việc: chỉ xét khoảng **2.000 vùng** "có thể có vật", hoặc nhìn cả ảnh **một lần** và đoán mọi vị trí cùng lúc — trang sau.

Hai bước hay một bước?

Hai bước (two-stage): R-CNN, Faster R-CNN



R-CNN: khoảng **2.000 vùng**, **mỗi vùng** chạy CNN một lần. Faster R-CNN: dùng chung đặc trưng, khoảng 300 vùng.

Một bước (one-stage): YOLO, SSD, RetinaNet

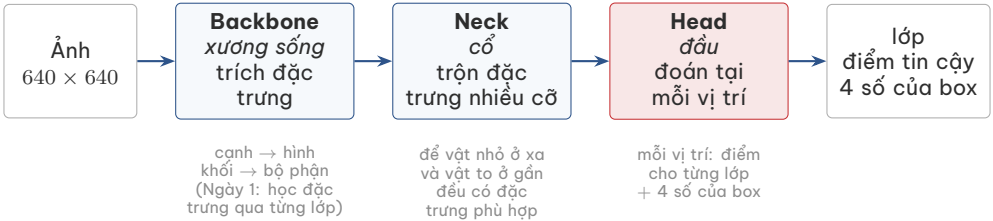


YOLO: chạy mạng **1 lần** cho cả ảnh — *You Only Look Once*

Nhanh hay chính xác?

Bảng 1 bài YOLOv1 (VOC 2007): Faster R-CNN VGG-16 **chính xác hơn** (73,2 so với 63,4 mAP; càng cao càng tốt), YOLO **nhanh hơn** (45 so với 7 ảnh/giây). Xe cần phát hiện kịp để phản ứng. Sau đó YOLOv2 vượt Faster R-CNN (ResNet) mà vẫn nhanh hơn (chương 5).

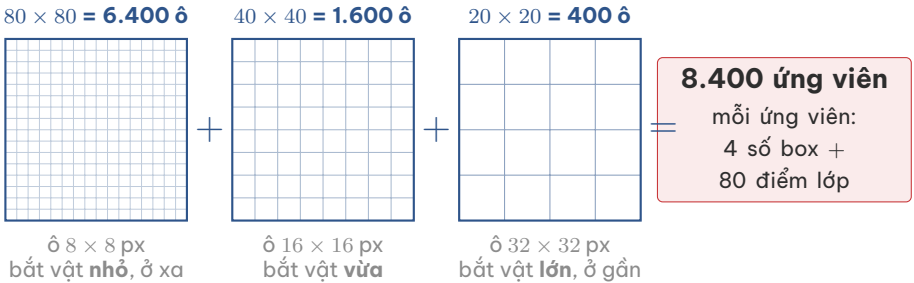
Detector kiểu YOLO có ba phần



Khi train, **cả ba phần** cùng được chỉnh cho khớp với box có nhãn. Đổi backbone là đổi "đôi mắt"; đổi head là đổi "cách trả lời".

Họ DETR (gặp ở chương 5) mô tả khác: backbone rồi bộ transformer encoder–decoder.

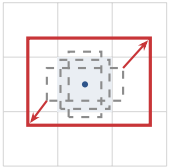
Một lần nhìn, 8.400 ứng viên



Ảnh 640×640 vào YOLO11 (bản đã học trên COCO, 80 lớp) $\rightarrow$ một bảng **8.400 dòng**, đoán cùng lúc. Gần hết là *nền* — bước tiếp theo là **lọc**.

Bốn số của box: có khung mẫu hay không?

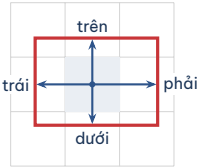
Có khung mẫu (anchor)



Mỗi ô **3 khung mẫu** đặt sẵn (nét đứt); **mỗi khung** sinh một box: **dịch** tâm, **co giãn**. YOLOv5 mặc định so khung mẫu với box trong dữ liệu trước khi train.

Ảnh 640 (YOLOv5): 8.400 ô $\times$ 3 = **25.200** ứng viên

Không khung mẫu (anchor-free)



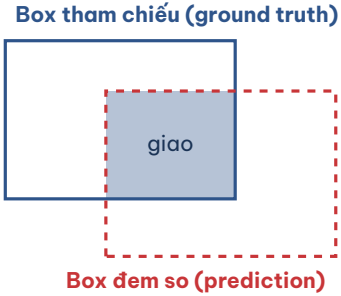
Từ **tâm ô**, suy ra **khoảng cách tới 4 cạnh** của box (YOLOv8, YOLO11, YOLO26). Không có khung mẫu nào phải khớp dữ liệu.

Ảnh 640 (YOLO11): mỗi ô 1 dự đoán = **8.400** ứng viên

Hai câu hỏi riêng: mấy bước, có khung mẫu?

	không khung mẫu	**có khung mẫu**
hai bước	R-CNN	Faster R-CNN (9 khung mẫu mỗi vị trí)
một bước	YOLOv1; YOLOX, YOLOv8, YOLO11, YOLO26	YOLOv2 đến YOLOv5

IoU: hai box chồng nhau bao nhiêu?



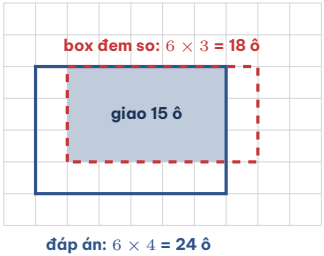
$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

$\text{IoU} = 1$ – trùng khít
$\text{IoU} = 0$ – không giao nhau

Quy ước chấm: đúng lớp và $\text{IoU} \geq 0.5$ thì tính là "khớp"

IoU chỉ đo **hai box chồng lên nhau bao nhiêu** – không biết bên trong là vật gì. Vì thế dùng được cho cả *model vs người* và *người vs người*.

Hoạt động: tính IoU bằng tay (30 phút)



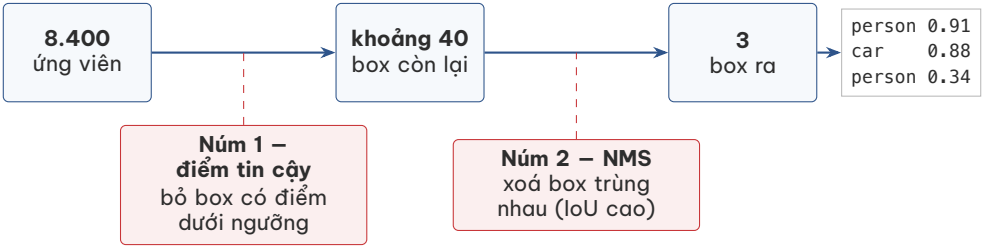
$$\text{hợp} = 24 + 18 - 15 = 27 \text{ ô}$$
$$\text{IoU} = 15/27 \approx \mathbf{0{,}56}$$

1. Ba ảnh in trên giấy kẻ ô, mỗi ảnh một chiếc xe **chưa có box**.
2. Tự vẽ box sát quanh xe — 20 phút, **chưa xem đáp án**.
3. Đặt tờ đáp án trong suốt lên. Đếm ô *giao*, ô *hợp*, rồi chia.
4. Cả lớp ghi số lên bảng: cùng một chiếc xe, IoU tản ra bao xa?

Đọc con số ví dụ

$0{,}56$ qua ngưỡng $0{,}5$ nhưng trượt mức kiểm tra $0{,}7$ dùng trong bài này. Không cần máy tính: đếm ô và một phép chia.

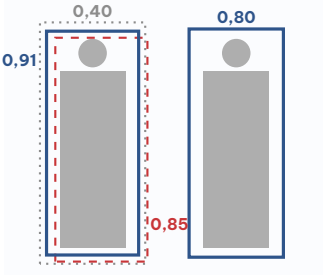
Từ 8.400 ứng viên còn vài box



Mặc định khi chạy `predict` của Ultralytics: `conf=0.25`, `iou=0.7`, tối đa `max_det=300` box.

Hạ ngưỡng điểm ⇒ thường thêm cả box thật *lẫn* box nhầm. Số 40 ở giữa là minh hoạ.

NMS: giữ box mạnh nhất, xoá box trùng nó

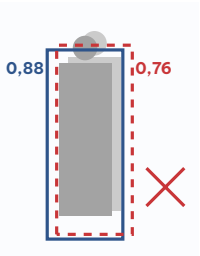


1. Xếp theo điểm: $0{,}91 \rightarrow 0{,}85 \rightarrow 0{,}80 \rightarrow 0{,}40$.
2. Giữ **0,91**. Xoá box trùng nó với IoU $> 0{,}7$: **0,85** (IoU 0,78) và **0,40** (IoU 0,81).
3. Box còn lại cao nhất là **0,80**: không chồng lên box đã giữ $\Rightarrow$ giữ. Đó là người thứ hai.
4. Kết quả: **2 box cho 2 người**.

NMS — Non-Maximum Suppression

Trong mỗi cụm box **cùng lớp** trùng nhau, giữ box điểm **cao nhất**. "Trùng" do ngưỡng IoU của NMS quyết định — ngưỡng lọc dự đoán này khác ngưỡng IoU khi chấm model.

Đám đông: NMS xoá nhầm, và cách bỏ NMS



IoU 0,74 > 0,7 ⇒ NMS xoá box 0,76

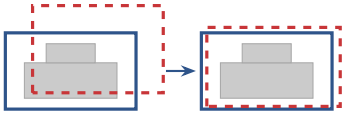
⇒ **sót một người** vì chính bước lọc

Hai cách xử lý

- **Nới ngưỡng IoU của NMS** (ví dụ 0,8): ít xoá nhầm hơn, nhưng sót lại nhiều box trùng hơn.
- **Model không cần NMS**: lúc train, mỗi vật chỉ được ghép với *một* dự đoán; dự đoán thừa bị phạt, nên model học cách không vẽ trùng. DETR (2020), YOLOv10 (2024), YOLO26 (2026; từ Ultralytics 8.4.142 cần `nms=False`).

Lưu ý: Bộ dữ liệu CrowdHuman có trung bình **22,6 người mỗi ảnh** — đúng loại cảnh NMS dễ xoá nhầm. Ngã tư xe máy chen nhau cũng vậy: đo detector trên cảnh **đông**, đừng chỉ cảnh thưa.

Model học vẽ box bằng cách nào?



- Mỗi box có nhãn được **ghép** với vài ứng viên ở gần nó.
- Ứng viên đoán **sai lớp** ⇒ bị phạt.
- Box **lệch**: IoU càng thấp, phạt càng nặng.
- Chỗ có vật mà dữ liệu **không có box** ⇒ ứng viên ở đó bị dạy rằng đó là **nền**.

Chuyên sâu: YOLO11 phạt box bằng CIoU + DFL, phạt lớp bằng BCE; YOLO26 bỏ DFL.

Học là lặp lại hàng nghìn lần

Đoán → so với box có nhãn → phạt → chỉnh tham số một chút (Ngày 1: đi xuống dốc sai số).

Detector: ba tầng hiểu

Ý tưởng: một detector đoán rất nhiều, rồi lọc bớt

•○○ **Cơ bản**	Ảnh vào → hàng nghìn ứng viên → bỏ ứng viên điểm thấp → xoá box trùng → vài box ra. Không có "nhìn thấy", chỉ có tính điểm và lọc.
••○ **Nâng cao**	YOLO11 (một bước, không khung mẫu): backbone trích đặc trưng, neck trộn nhiều cỡ, head đoán trên 3 lưới (80, 40, 20 ô mỗi chiều với ảnh 640). Loss = sai lớp + lệch box. Hai núm khi chạy: `conf` và `iou` của NMS.
••• **Chuyên sâu**	Phần khó nhất là **ghép**: ứng viên nào học từ box nào. Ghép một-nhiều thì lúc chạy phải có NMS; ghép một-một (DETR, YOLOv10, YOLO26) bỏ được NMS nhưng *không* bỏ được ngưỡng điểm tin cậy.

Đo Detector Và Đo Dữ Liệu

Precision, recall, mAP — và đo chính dữ liệu

Từ IoU đến ba loại kết quả

- **True positive (TP)** — đúng lớp, đủ IoU, ghép với một vật chưa được ghép.
- **False positive** — báo có nhưng không khớp ground truth nào, kể cả box **thứ hai** cho một vật đã khớp.
- **False negative** — có ground truth, không prediction nào khớp.

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

Đọc bằng tiếng Việt

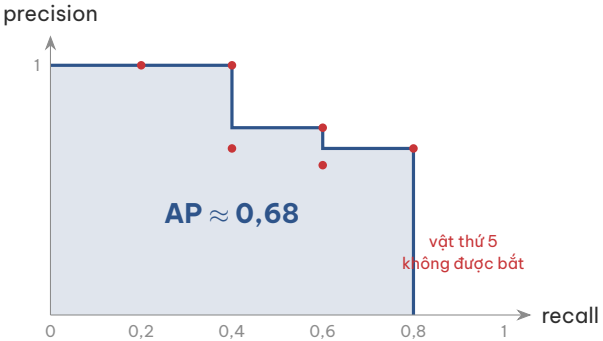
Precision — *model báo, bao nhiêu là thật?* **Recall** — *thật sự có, model bắt được bao nhiêu?*
Với người đi bộ trước xe, **recall thấp** thường nguy hiểm hơn — nhưng phanh nhầm cũng có giá.

Đường precision–recall và AP

Ảnh có **5** vật có box. Model trả **6** dự đoán:

Điểm	Khớp?	P	R
0,95	TP	1,00	0,2
0,90	TP	1,00	0,4
0,80	FP	0,67	0,4
0,70	TP	0,75	0,6
0,55	FP	0,60	0,6
0,40	TP	0,67	0,8

Đi từ điểm cao xuống: TP đẩy recall lên, FP kéo precision xuống.



AP — diện tích dưới đường bậc thang

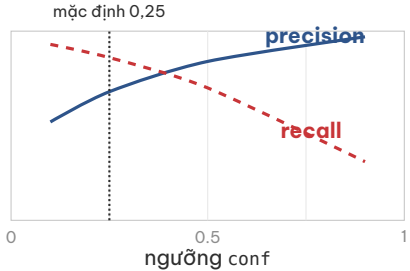
Mỗi lớp một đường, một AP. Vật **không bao giờ** được phát hiện làm đường dừng ở recall 0,8 — phần bên phải là điểm mất hẳn. COCO lấy trung bình 101 điểm trên đường này: 0,69.

AP và mAP — một con số cho cả dataset

- **AP** — gộp precision + recall của *một lớp* thành một số.
- **mAP** — trung bình AP của *mọi lớp*. Một lớp có dữ liệu kém kéo cả mAP xuống — xem AP từng lớp.
- **mAP50** — ngưỡng $\mathrm{IoU} = 0.5$. Đọc thô: *"có tìm ra vật thể không"*.
- **mAP50-95** — trung bình 10 ngưỡng. Đọc thô: *"box có bám sát không"* — cả hai vẫn tính cả sai lớp, bỏ sót, box trùng.

Lưu ý: mAP50 cao mà mAP50-95 thấp: box **chưa đủ sát** để qua ngưỡng khắt khe. Có thể do model, do vật quá nhỏ, hoặc do chính box trong dữ liệu — phải mở ảnh ra xem.

Đổi ngưỡng: được gì, mất gì?



Đường cong minh hoạ, không phải kết quả chạy thật.

- **Hạ ngưỡng**: giữ cả box điểm thấp ⇒ recall *không giảm*, thường tăng; precision *thường* giảm vì thêm báo nhầm. **Nâng ngưỡng**: ngược lại.

Chọn theo việc, không theo số đẹp

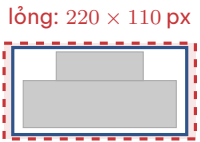
Lọc ảnh cho người xem lại: ngưỡng thấp.

Cảnh báo cho tài xế: báo nhầm nhiều thì bị tắt, bỏ sót thì nguy hiểm.

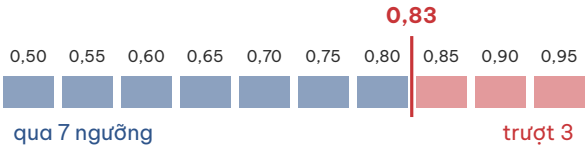
Chiều nay, trạm 4

Chạy lại `predict` với `conf=0.1 / 0.25 / 0.5 / 0.9` trên cùng ảnh giữ lại; đếm đúng, bỏ sót, báo nhầm.

Box lỏng 10% vẫn qua IoU 0,5



$$\text{IoU} = \frac{20.000}{24.200} \approx \mathbf{0{,}83}$$



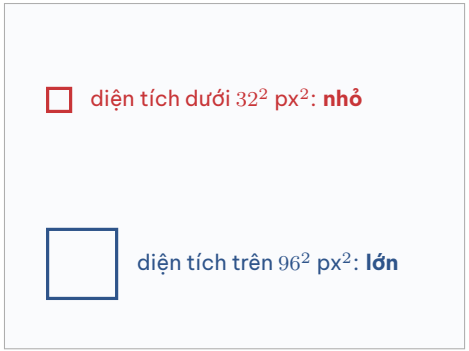
mAP50-95 chấm ở 10 ngưỡng IoU; box này qua 7, trượt 3.

So với box sát	IoU	QC 0,7
mỗi chiều $\times 1{,}1$	$1/1{,}21 \approx 0{,}83$	qua
mỗi chiều $\times 1{,}25$	$1/1{,}5625 = 0{,}64$	**trượt**

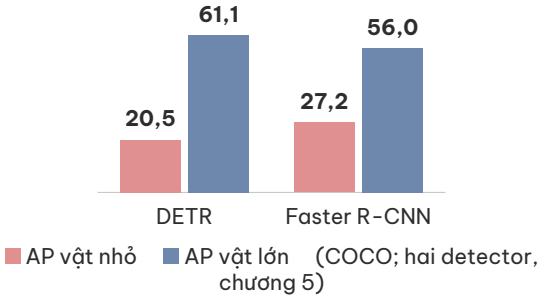
Đoán trước cho chiều nay

Nếu mọi box train rộng và cao thêm 10% còn box kiểm tra vẫn sát: **mAP50** thay đổi ít, **mAP50-95** giảm. Lab kiểm chứng bằng bảng kết quả tính sẵn cho sát / $+10\%$ / $+25\%$.

Một con số mAP che ba cỡ vật



ảnh 640×480, vẽ đúng tỉ lệ



(COCO; hai detector, chương 5)

Cùng model, cùng dữ liệu, chênh ba lần

COCO chấm riêng AP cho vật nhỏ, vừa, lớn. AP vật nhỏ của DETR chỉ bằng **một phần ba** AP vật lớn — nhiều xe máy ở xa thì đọc AP theo cỡ.

IoU và mAP: ba tầng hiểu

Ý tưởng: mAP gộp hàng nghìn lần so box thành một số — tiện để so, nhưng không nói sai ở đâu

●○○ **Cơ bản**	IoU đo hai box trùng nhau bao nhiêu, từ 0 đến 1. Dự đoán được tính là đúng khi đúng lớp và IoU qua ngưỡng. mAP càng cao càng tốt — nhưng muốn biết sai ở đâu phải mở ảnh ra xem.
●●○ **Nâng cao**	AP là diện tích dưới đường precision–recall của một lớp; mAP là trung bình các lớp. mAP50 dùng ngưỡng IoU 0,5; mAP50-95 lấy trung bình 10 ngưỡng. P và R mà Ultralytics in ra được đọc ở *một* ngưỡng điểm chung cho mọi lớp.
●●● **Chuyên sâu**	Khi ghép dự đoán với box có nhãn, COCO duyệt dự đoán theo điểm từ cao xuống; Ultralytics ưu tiên cặp IoU cao nhất, rồi mới xếp theo điểm để vẽ đường PR và tính diện tích bằng hình thang — hai công cụ có thể cho số hơi khác. Confusion matrix vẽ ở `conf` 0,25 và IoU 0,45.

Kiểm tra độ nhất quán của dữ liệu

Hai bộ box tham chiếu do hai người tạo cho **cùng 20 ảnh**, ghép từng cặp để so:

IoU trung bình thấp	Kiểm quy tắc box (sát, bị che, bị cắt)
Ít thống nhất về lớp	Kiểm định nghĩa lớp
Nhiều box lẻ	Kiểm phạm vi cần tìm

Gold set: đáp án chốt; **Consensus**: đối chiếu;
Review sampling: kiểm mẫu. Chi tiết: Ngày 06.

> **Lưu ý:** mAP đo model $\neq$ số kiểm dữ liệu.

Bộ dữ liệu nổi tiếng cũng có box sai

VINUNIVERSITY

Bỏ sót

Bỏ ngẫu nhiên **30%** box trong dữ liệu train: mAP50 chỉ giảm khoảng **5%**, giảm nhiều hơn ở ngưỡng IoU khắt khe. Con số quen nhìn không báo động.

Vẽ lệch

COCO-ReM vẽ lại đường viền vật trong COCO cho sát hơn: model được chấm cao hơn **chủ yếu ở IoU** $\geq 0{,}75$, gần như không đổi ở 0,5.

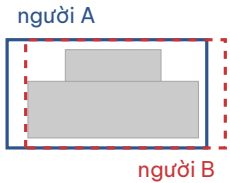
Khác kiểu vẽ

Cùng ảnh COCO, hai đội vẽ theo hai hướng dẫn. Train theo kiểu này, chấm theo kiểu kia: mất **1–2 điểm mAP**.

Ba loại lỗi, và cách tìm

Bỏ sót, lệch chỗ, nhầm lớp (cách gọi của ObjectLab, 2023). Công cụ tìm lỗi dùng chính dự đoán của model để chỉ ra *ảnh nào nên xem lại* — còn sửa hay không vẫn do người quyết định.

Thảo luận: IoU 0,83 — cùng một box hay bất đồng?



IoU = **0,83**

1. Hai người cẩn thận, IoU 0,83: **cùng một box**, hay một bất đồng cần sửa? Ai vạch ranh giới?
2. Nếu 0,83 là ổn, vì sao bài này dùng mức kiểm tra **0,7** chứ không phải 0,5?
3. Chênh lệch cỡ 0,83 **tốn gì** cho model? Chiều nay bảng sát / $+10\%$ / $+25\%$ trả lời.

Một mốc để so

Hai chuyên gia vẽ lại cùng vật trên PASCAL VOC: IoU trung bình **0,88**; **93%** cặp box trên 0,7. Không ai vẽ trùng khít.

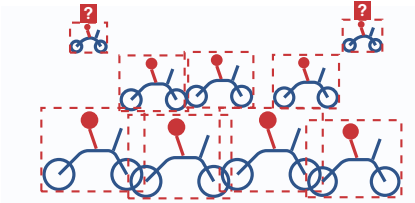
Câu nào chưa trả lời được: ghi lên bảng, mang sang lab.

Dữ Liệu Dạy Detector

04

Quy tắc nào quyết định model học được gì

Đường phố Việt Nam: xe máy chen nhau



Xe gần, xe xa, xe che nhau: bao nhiêu box?

Guideline phải trả lời trước khi thu ảnh

- **Xe che nhau**: mỗi xe một box riêng, kể cả khi chỉ thấy nửa xe — đánh `occluded`.
- **Hai, ba người trên một xe**: vẽ mấy box? Bộ dữ liệu đường phố Ấn Độ IDD ghi nhận cảnh này rất phổ biến.
- **Xe ở xa, vài chục pixel**: nhỏ dưới bao nhiêu thì bỏ? Cùng lệch 5 px, box 20×40 còn IoU **0,60**; box 200×100 vẫn **0,95**.
- **Khi đo**: tách riêng nhóm ảnh đông — chỗ NMS dễ xoá nhầm.

Vì sao phải chốt trước

Cùng một ảnh, người này vẽ 9 box, người kia vẽ 6: model nhận **hai định nghĩa** khác nhau về "chiếc xe nào cần có box".

Bốn việc, theo đúng thứ tự

1. **Class design** — chốt danh sách lớp *trước* khi vẽ.
2. **Vẽ box** — mỗi vật thể một box, bám sát và đúng lớp.
3. **Attribute** — `occluded`, `truncated`, `parked`.
4. **Export** — xuất sang định dạng model đọc được.

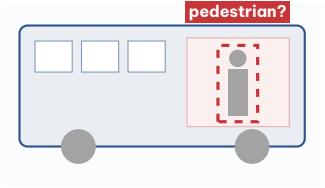
Class hay attribute?

> **Class** = vật thể *là gì*. **Attribute** = *trạng thái* của nó. Xe đang đỗ vẫn là `car`.

Tách lớp hay thêm thuộc tính?

Hai thứ trông khác nhau	1. Phía sau có xử lý chúng khác nhau không?	2. Nhìn một ảnh có phân biệt chắc không?	Chọn
xe buýt / ô tô con	có — to hơn, hay dừng đón khách	có	**hai lớp**
ô tô đang đỗ / đang chạy	có — đứng yên hay đang di chuyển	không phải lúc nào — ảnh tĩnh không có chuyển động	**một lớp** + thuộc tính `parked`
xe tải đỏ / xe tải xanh	không	có	**một lớp**, không cần thuộc tính

*Chỉ tách thành hai lớp khi **cả hai** câu trả lời là "có". Mỗi lớp thêm vào là thêm một chỗ để hai người nhìn cùng một ảnh mà chọn khác nhau.*



Hình người in trên thân xe buýt

Guideline ví dụ, tự đặt cho bài học

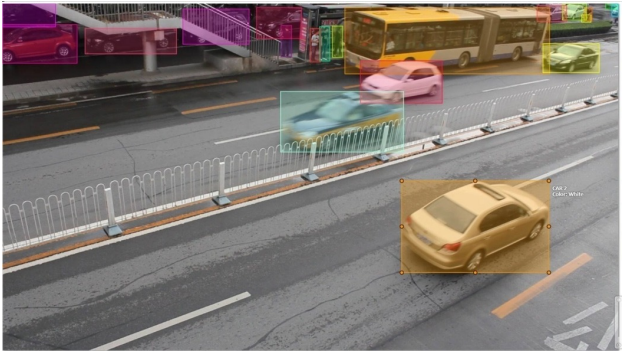
Mục 2.1 — Gán `pedestrian` cho *mọi hình người*.
Mục 4.3 — Chỉ gán vật *tự di chuyển được*.

Theo 2.1: có box. Theo 4.3: không box. Cả hai cách đều "làm đúng guideline".

1. Ghi lại: ảnh, hai mục mâu thuẫn, hai cách hiểu.
2. **Gắn cờ** ca đó — không tự chọn một bên.
3. Người giữ guideline quyết **một lần**; cả nhóm theo.

Lưu ý: Mỗi người tự chọn một kiểu thì cùng một loại hình ảnh vừa là `pedestrian` vừa là nền trong dữ liệu train: model nhận hai tín hiệu ngược nhau.

Dữ liệu dạy detector trông như thế nào?



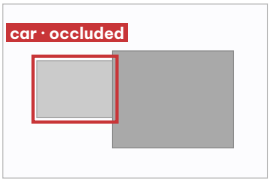
Dữ liệu tham chiếu: mỗi xe một box **bám sát** thân xe, không thừa mặt đường.

- Xe sau hàng rào vẫn có box ⇒ đánh `occluded`
- Xe rất nhỏ ở rìa trên ⇒ hỏi guideline: gán hay bỏ?
- Màu box = lớp; box đang chọn có tay cầm để kéo lại cho khít

Chữ trong ảnh quá nhỏ để đọc khi chiếu — hãy nhìn **hình khối** của các box.

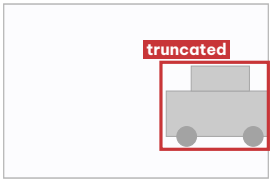
Nguồn: docs.cvat.ai — MIT; ảnh từ bộ dữ liệu DETRAC

Ba ca khó nhất khi vẽ box



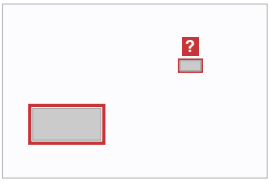
Bị che

Theo quy tắc ví dụ: box phần nhìn thấy, đánh dấu `occluded`



Bị cắt ở rìa

Box dừng ở rìa ảnh, không đoán phần ngoài



Nhỏ / ở xa

Dưới ngưỡng trong guideline thì bỏ — đừng tự quyết

Cả ba ca do **guideline** trả lời trước. Mọi box: **sát** vật (tight), **đủ** mọi vật thuộc lớp cần tìm; ca mơ hồ thì ghi lại và hỏi.

Tight box không phải chuyện thẩm mỹ

Box bám sát



Box ôm đúng mép vật ⇒ model có *đích* đúng để học cỡ và vị trí.

Box lỏng



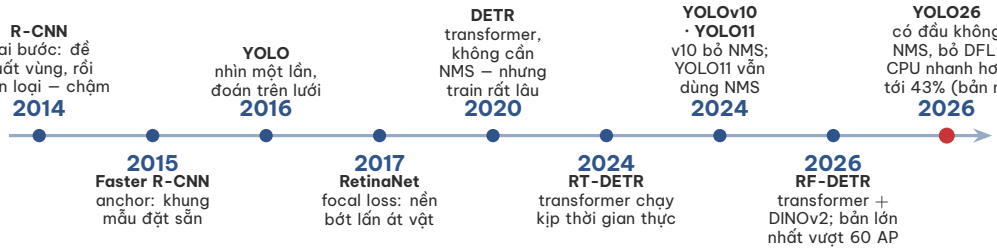
Đích học to hơn vật thật ⇒ box lúc chạy **có thể to theo** ⇒ bước nào dùng cỡ box (ví dụ ước lượng khoảng cách) dễ nhận số sai.

05

Họ YOLO Và Detection 2026

Mười một năm YOLO, và chọn model nào năm 2026

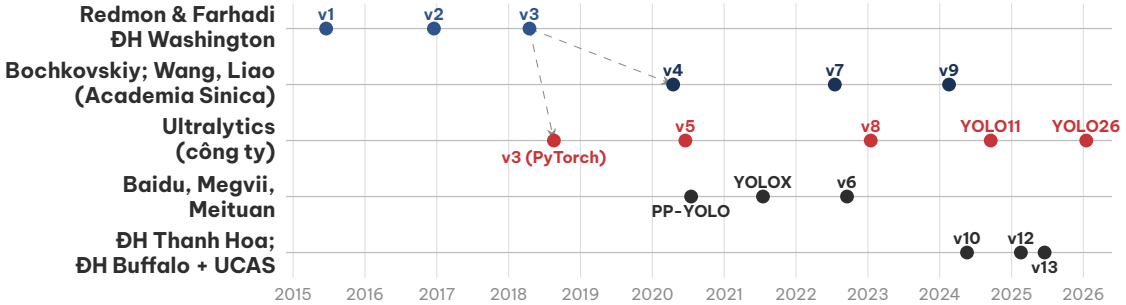
Detector qua từng thế hệ



Kiến trúc thay đổi liên tục. Điều không đổi: model nào cũng cần **ảnh có box** — ít nhất để **chấm điểm**, và thường cả để học.

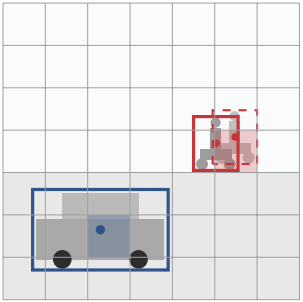
Năm: năm hội nghị công bố; YOLO11 và YOLO26 là năm phát hành. Riêng họ YOLO: các trang sau.

Một cái tên, nhiều nhóm tác giả



Tên YOLO được **ít nhất tám nhóm** dùng (gộp thành năm làn). Số phiên bản là tên nhóm tự đặt — nhóm YOLOv6 viết rằng họ đặt tên "với sự cho phép của các tác giả YOLO" — **không phải bảng xếp hạng**: YOLOv13 (06/2025) ra *trước* YOLO26 (01/2026).

YOLOv1 (2015): chia ảnh thành lưới 7×7



Ô xanh chứa tâm ô tô: ô này đoán ô tô.

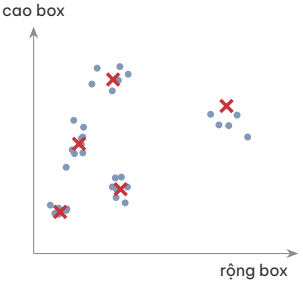
Ô đỏ chứa tâm của **hai** xe máy.

- Ảnh thu về 448×448, chia **lưới 7×7**. Tâm vật rơi vào ô nào, **ô đó chịu trách nhiệm** đoán vật.
- Mỗi ô: **2 box** (x, y, w, h, điểm) và điểm 20 lớp $\Rightarrow$ $7 \times 7 \times (2 \times 5 + 20) = \mathbf{1.470\ số}$.
- Bài báo tự nêu điểm yếu: mỗi ô chỉ 2 box, 1 lớp $\Rightarrow$ khó với vật nhỏ đi thành nhóm ("đàn chim"; ngoài phố: xe máy chen nhau).
- Titan X: **45 ảnh/giây**, 63,4 mAP (VOC 2007). Faster R-CNN: 73,2 mAP, chỉ 7 ảnh/giây.

Box đủ mà model vẫn sót

Dữ liệu có box cho từng xe, nhưng v1 **giới hạn** số vật sát nhau đoán được trong một ô. Đọc lỗi phải hỏi: dữ liệu thiếu, hay model không có chỗ để đoán?

YOLOv2 (2016): khung mẫu lấy từ chính dữ liệu



Chấm: cỡ một box trong tập train.

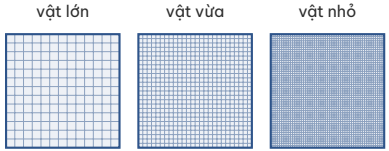
×: 5 tâm cụm = 5 khung mẫu (minh hoạ).

- **Khung mẫu** (chương 2), YOLOv2 mượn từ Faster R-CNN: mỗi ô chỉnh nhiều khung có sẵn.
- Khung lấy **từ dữ liệu**: k-means trên cỡ box của tập train (khoảng cách $1 - \mathrm{IoU}$), $k = 5$. Trên VOC, COCO: khung nghiêng về **cao, hẹp**.
- Số box đoán: 98 $\rightarrow$ hơn 1.000; recall 81% $\rightarrow$ 88%.
- Train xen nhiều cỡ ảnh. Ảnh 416: 76,8 mAP (VOC 2007), 67 ảnh/giây (Titan X).

Dữ liệu định hình model trước cả khi train

Khung mẫu tính từ box của tập train: cỡ hay tỉ lệ box đổi nhiều thì nên kiểm tra lại; box train to hơn vật một cách có hệ thống thì khung mẫu to theo.

YOLOv3 (2018): ba cỡ lưới, chín khung mẫu



$(13^2 + 26^2 + 52^2) \times 3 = \mathbf{10.647}$ box

ảnh 416, 3 khung mẫu mỗi ô

Chương 2: 8.400 ở ảnh 640 — không khung mẫu, mỗi ô 1 dự đoán

- Đoán ở **3 cỡ lưới**: lưới mịn bắt vật nhỏ, lưới thô bắt vật lớn. 9 khung mẫu từ k-means trên COCO, từ 10×13 đến 373×326 px.
- Mỗi box thật chỉ gán cho **một** khung mẫu khớp nhất.
- **Nhiều nhãn cho một vật**: bỏ softmax, mỗi lớp tự trả lời có hay không — một người có thể vừa "Person" vừa "Woman" (Open Images).
- COCO, ảnh 608: **57,9 AP50 trong 51 ms** (Titan X); RetinaNet 57,5 AP50 trong 198 ms.

Câu hỏi ở cuối chính báo cáo YOLOv3

Làm ra detector rồi, người ta sẽ dùng nó vào việc gì? Tác giả nhắc tới thu thập dữ liệu cá nhân và quân sự, và viết: người nghiên cứu có trách nhiệm **cân nhắc tác hại** của công việc mình.

2020: YOLOv4 và YOLOv5 — cùng năm, hai nhóm

YOLOv4 (04/2020): một bài báo

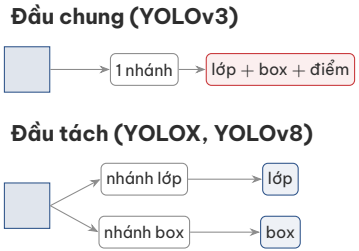
- **Bag of freebies**: chỉ đổi cách train, chạy không chậm thêm (mosaic, loss CIoU).
- **Bag of specials**: tốn thêm chút tính toán lúc chạy, được thêm độ chính xác (SPP, PAN).
- COCO: 43,5 AP50-95, khoảng 65 ảnh/giây (V100).

YOLOv5 (06/2020): một thư viện

- Ultralytics viết bằng PyTorch: cài, train, export chỉ vài lệnh ⇒ rất nhiều người dùng.
- **Không có bài báo**: muốn biết chi tiết phải đọc mã và tài liệu.
- Giấy phép hiện nay: AGPL-3.0.

Lưu ý: Mosaic (ghép 4 ảnh) là của ai? Bài YOLOv4 gọi là "phương pháp mới"; bài YOLOX ghi là do Ultralytics YOLOv3 đề xuất. Hai nguồn ghi khác nhau — chỉ chắc một điều: mosaic đổi **ảnh train**, không đổi model.

YOLOX (2021) đến YOLOv8 (2023): bỏ khung mẫu



- **Không khung mẫu** (chương 2; YOLOX, Megvii): mỗi vị trí đoán thẳng 1 box thay vì chỉnh 3 khung ⇒ bỏ bước k-means.
- **Tách đầu**: nhánh lớp riêng, nhánh box riêng — bài báo: học nhanh hơn hẳn.
- **Ghép động** (SimOTA): số ô học từ một box tuỳ ô nào đang đoán tốt.
- **YOLOv8** (Ultralytics, 01/2023) gom lại: không khung mẫu, tách đầu, ghép TAL, loss CIoU + DFL. Cùng giai đoạn: YOLOv6 (Meituan), YOLOv7 (Academia Sinica).

Bỏ khung mẫu, không bỏ được box

Không còn khung tính từ dữ liệu, nhưng box thật vẫn quyết định **ô nào được học**: box lệch thì ô được chọn lệch theo.

2024: YOLOv9 giữ thông tin, YOLOv10 bỏ NMS

YOLOv9 (02/2024)

- Qua nhiều tầng, mạng **rơi dần thông tin** về ảnh gốc ⇒ tín hiệu học yếu đi.
- **PGI**: thêm các nhánh phụ *chỉ dùng lúc train* để tín hiệu học (gradient) đủ tin cậy; lúc chạy chỉ giữ nhánh chính, model không nặng thêm.
- **GELAN**: khối kiến trúc nhẹ đi kèm.

YOLOv10 (05/2024)

- NMS làm chậm và khó đóng gói model trọn vẹn.
- Train **hai đầu**: một-nhiều (mỗi vật nhiều ô) và một-một (mỗi vật đúng một ô), ghép bằng **cùng một thước**.
- Lúc chạy chỉ dùng đầu một-một ⇒ **không cần NMS**.

Không còn NMS dọn giúp

Đầu một-một học mỗi vật một box. Nếu một vật bị ghi hai box trong dữ liệu train, model có thể học ra hai box — và lúc chạy không còn NMS gộp giúp.

YOLO11, YOLO12, YOLO26: dòng đang dùng năm 2026

VINUNIVERSITY

YOLO11 (09/2024)

Ultralytics tinh chỉnh YOLOv8: YOLO11m ít hơn **22%** tham số mà mAP (COCO) cao hơn YOLOv8m. Vẫn cần NMS. Lab chiều nay dùng `yolo11n.pt`.

YOLO12 (02/2025)

ĐH Buffalo và UCAS đưa **attention** vào lõi. Ultralytics ghi: model cộng đồng, để so sánh và nghiên cứu; triển khai thì chọn YOLO11 hoặc YOLO26.

YOLO26 (01/2026)

Hai đầu như YOLOv10, bỏ DFL. Train mới: MuSGD (optimizer từ LLM), Progressive Loss (dồn dần sang đầu một-một), STAL (vật nhỏ luôn có ô học). CPU nhanh hơn YOLO11n tới **43%** (bản n, ONNX).

Mặc định đổi theo phiên bản: Ultralytics 8.4.0–8.4.141 chạy YOLO26 bằng đầu không NMS; từ 8.4.142 mặc định qua NMS, phải thêm `nms=False`. Kiểm tra bản đang cài trước khi đo tốc độ.

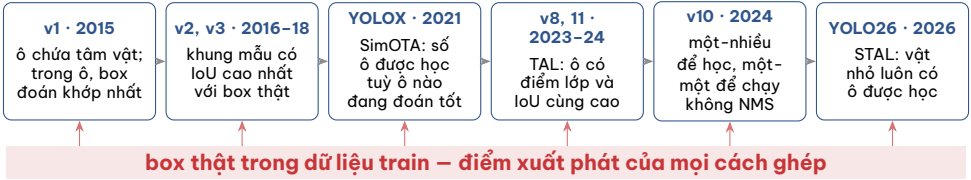
Lưu ý: YOLOv13 (06/2025, ĐH Thanh Hoa) **không** thuộc Ultralytics và ra *trước* YOLO26: số to hơn không có nghĩa là mới hơn hay tốt hơn.

Vì sao không xếp hạng các đời YOLO bằng một cột số

Model (ảnh vào)	Năm	Con số	Thước đo	Dữ liệu	Phần cứng, tốc độ
YOLOv1 (448)	2015	63,4	mAP50	VOC 2007	Titan X, 45 ảnh/s
YOLOv2 (416)	2016	76,8	mAP50	VOC 2007	Titan X, 67 ảnh/s
YOLOv3 (608)	2018	57,9	mAP50	COCO	Titan X, 51 ms
YOLOv4	2020	43,5	mAP50-95	COCO	V100, ≈65 ảnh/s
YOLOX-L	2021	50,0	mAP50-95	COCO	V100, 68,9 ảnh/s
YOLOv7-E6E (1280)	2022	56,8	mAP50-95	COCO	V100, ≥30 ảnh/s
YOLO26 n–x (640)	2026	40,1–56,9	mAP50-95	COCO	T4, 1,7–11,8 ms

Hai thước đo, hai bộ dữ liệu (VOC 2007 dễ hơn COCO nhiều), ba loại GPU, nhiều cỡ ảnh và cỡ model; YOLO26 đo ở chế độ không NMS. Đọc cột số từ trên xuống thì tưởng YOLO kém dần — thật ra các số không so được với nhau. So hai model: cùng bộ dữ liệu, cùng thước đo, cùng phần cứng (chương 3).

Sợi chỉ xuyên suốt: ô nào học từ box nào?



Mười một năm, sáu chặng tiêu biểu của cách ghép; chỉ **đầu vào** không đổi: box thật.

Vì sao chuyện ghép liên quan tới dữ liệu?

Cách ghép nào cũng đọc **box thật** trước tiên. Box lệch, box sót, box ghi trùng làm sai chuyện **ô nào được học** — dù model đời nào. Chương 3 đo hậu quả; ở đây là nguyên nhân.

Lệnh train chiều nay mang dấu vết 11 năm YOLO

VINUNIVERSITY

Trong lab	Đến từ	Liên quan tới dữ liệu
`imgsz=640`: ba lưới 80, 40, 20	nhiều cỡ lưới: YOLOv3 (2018)	vật nhỏ nằm ở lưới mịn nhất; box nhỏ lệch vài pixel là IoU tụt mạnh
`mosaic=1.0`	Ultralytics YOLOv3 hay YOLOv4 (2020)? hai bài báo ghi khác nhau	box bị cắt theo mép ảnh ghép; vật gặp ở nhiều cỡ, nhiều nền
`close_mosaic=10`	YOLOX (2021): tắt tăng cường mạnh ở cuối	tắt mosaic ở 10 epoch cuối
không khung mẫu	YOLOX (2021), YOLOv8 (2023)	không phải tính khung mẫu theo từng bộ ảnh
ghép ô với box bằng TAL	TOOD (2021)	box thật quyết định ô nào được học
loss box CIoU + DFL (hệ số `box=7.5`, `dfl=1.5`)	CIoU (2019), DFL (2020)	đích học chính là box trong dữ liệu — box lỏng thì đích lỏng
đổi sang `yolo26n.pt`: hai đầu, bỏ DFL	YOLOv10 (2024), YOLO26 (2026)	chạy không NMS thì không còn bước dọn box trùng

Giá trị mặc định: `default.yaml` của Ultralytics bản 8.4. Năm: bản arXiv đầu tiên hoặc năm phát hành.

Họ YOLO: ba tầng hiểu

Ý tưởng: các bản YOLO, từ nhiều nhóm, sửa những điểm yếu khác nhau của detector trước đó — và bản nào cũng học từ box trong dữ liệu

●○○ **Cơ bản**	YOLO = nhìn ảnh một lần, đoán box trên lưới. Số phiên bản là **tên** do từng nhóm đặt, không phải thứ hạng.
●●○ **Nâng cao**	Lưới 7×7 (v1) → khung mẫu từ k-means, 3 cỡ lưới (v2, v3) → bỏ khung mẫu, tách đầu (YOLOX, YOLOv8) → đầu một-một, bỏ được NMS (YOLOv10, YOLO26). Đọc số: cùng bộ dữ liệu, cùng thước đo, cùng phần cứng.
●●● **Chuyên sâu**	Nhiều bước tiến lớn chỉ đổi **lúc train** (freebies, PGI, cách ghép ô với box) — lúc chạy model không nặng thêm. Đổi cách ghép là đổi cách box trong dữ liệu tác động lên model: ghép một-một thì không còn NMS che box trùng.

Detector thời gian thực 2026: CNN hay transformer?

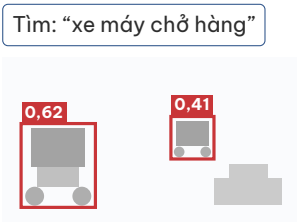
Model	Kiểu	COCO mAP50-95	Tham số	T4 (ms)	Giấy phép
YOLO11n	CNN	39,5	2,6 M	1,5	AGPL-3.0
YOLO11s	CNN	47,0	9,4 M	2,5	AGPL-3.0
YOLO26n	CNN, không NMS	40,1	2,4 M	1,7	AGPL-3.0
YOLO26s	CNN, không NMS	47,8	9,5 M	2,5	AGPL-3.0
RT-DETRv2-S	transformer	48,1	20 M	4,6	Apache-2.0
D-FINE-S	transformer	48,5	10,2 M	3,5	Apache-2.0
RF-DETR Nano (384 px)	transformer	48,4	30,5 M	2,3	Apache-2.0
RF-DETR Small (512 px)	transformer	53,0	32,1 M	3,5	Apache-2.0

Mỗi dòng do chính nhóm làm model tự đo, cài đặt khác nhau (ảnh vào: YOLO, RT-DETR 640 px; RF-DETR 384/512 px). Cùng YOLO11-S trên T4, ba nguồn báo 2,5 / 6,81 / 7,05 ms: so trong cùng một nguồn, đừng xếp hạng chéo. YOLO26 đo ở chế độ không NMS (`nms=False`).

Chọn model theo bài toán, không theo bảng xếp hạng

Nếu bài toán…	thì thử…	vì…
chạy trên CPU, camera, thiết bị nhỏ	bản *nano*: YOLO11n, YOLO26n	ít tham số; YOLO26n chạy CPU nhanh hơn YOLO11n tới 43%
cần chính xác nhất, có GPU	bản lớn hơn, hoặc họ DETR	thường mAP cao hơn, đổi lại chậm hơn
cảnh đông, box chồng lên nhau	model không cần NMS (YOLO26: `nms=False`)	không xoá nhầm vì bước NMS
có lớp mà COCO không có	model pretrained, train tiếp trên ảnh của mình	chiều nay làm đúng việc này
đưa vào sản phẩm	đọc giấy phép **từng checkpoint**	AGPL-3.0 có thể buộc cung cấp mã nguồn, kể cả khi chỉ phục vụ qua mạng — hỏi pháp chế; DEIMv2 cấm dùng thương mại nếu chưa mua phép

Gõ chữ, ra box: open-vocabulary

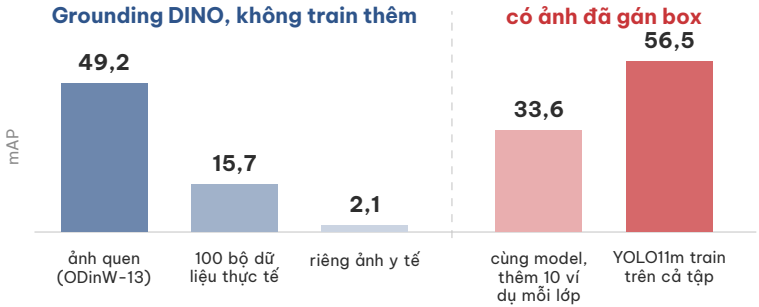


- Không cần train lại cho từng lớp: **gõ tên**, model tự tìm.
- **Grounding DINO**: 52,5 AP trên COCO dù *không* train bằng ảnh COCO.
- **YOLO-World**, **YOLOE**: kiểu YOLO, chạy nhanh; YOLOE có sẵn trong Ultralytics.
- **SAM 3** (11/2025): tìm *mọi* vật khớp một cụm từ, trả cả box; đạt khoảng **75%** mức của người theo box cgF1 trên SA-Co/Gold, bộ thử của Meta.

Năm 2026 dùng để làm gì

Vẽ nháp nhanh, lọc ảnh có vật hiếm, thử ý tưởng trước khi thu dữ liệu. Chưa thay được model train trên dữ liệu của chính bài toán — slide sau.

Zero-shot: giỏi ở ảnh quen, tụt ở ảnh lạ



Trên cảnh lái xe hiếm gặp (OpenAD), zero-shot chỉ **8,5–10,2 AP**; model train đúng dữ liệu lái xe đạt **16,2**.
Muốn giỏi ở đường phố của mình, vẫn cần **ảnh của mình, có box đã kiểm tra**.

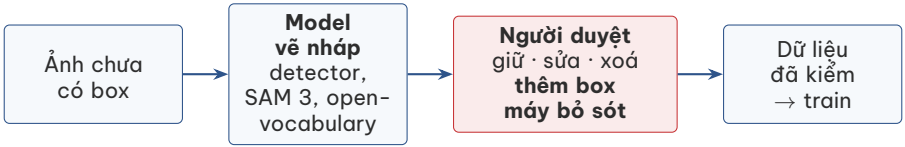
Nguồn: Roboflow100-VL (2025); OpenAD (CVPR 2025).

Chatbot cũng vẽ box — đọc kỹ bốn con số

Nguồn	Thứ tự	Thang đo	Cùng một box
File nhãn YOLO	c_x, c_y, w, h	0–1	0.5 0.5 0.1 0.2
File COCO	x, y, w, h (góc trên-trái)	pixel	864 432 192 216
Gemini	$y_{\min}, x_{\min}, y_{\max}, x_{\max}$	0–1000	400 450 600 550
Qwen2.5-VL	x_1, y_1, x_2, y_2	pixel	864 432 1056 648
Qwen3-VL	x_1, y_1, x_2, y_2	0–1000	450 400 550 600

Cùng một chiếc xe, năm dãy số. Dán nhầm quy ước thì box bị lộn trục hoặc sai tỉ lệ — không có thông báo lỗi nào. Chatbot còn thường không trả điểm tin cậy cho từng box.

Máy vẽ nháp, người duyệt: box năm 2026



Vì sao làm vậy

Vẽ một box từ đầu: khoảng **25 giây** (trung vị). Bấm "đúng/sai" cho một box máy vẽ: khoảng **1,6 giây**. Tổng thời gian giảm **6–9 lần**.

Cái giá phải để ý

Nháp tệ kéo cả người giỏi sai theo: một nghiên cứu (văn bản, 2010) thấy người giỏi nhất tụt **98,1% → 95,8%**. Box máy *không vẽ* là lỗi khó thấy nhất.

Công cụ 2026: CVAT có SAM và model tự vẽ nháp; Ultralytics Platform có "Smart Annotation". Hàm `auto_annotate` của Ultralytics ghi ra *polygon*, không ghi box.

Thảo luận (10 phút): chọn model nào, mượn ý tưởng nào?

1. Ngã tư giờ tan tầm

Xe máy chen nhau, box chồng lên nhau. Vì sao NMS có thể xoá nhầm? Ý tưởng nào trong chương giúp được, và dữ liệu cần gì?

2. Biển báo nhỏ, máy chỉ có CPU

Chọn bản nào? Tăng `imgsz` được gì, mất gì? Đo bằng con số nào của chương 3?

3. Đưa vào sản phẩm đóng mã

Chọn YOLOX, YOLOv7 hay YOLO11? Giấy phép *hiện tại* của từng repo nói gì, và ai quyết định?

4. Repo tên "YOLOv14" hứa $+5$ mAP

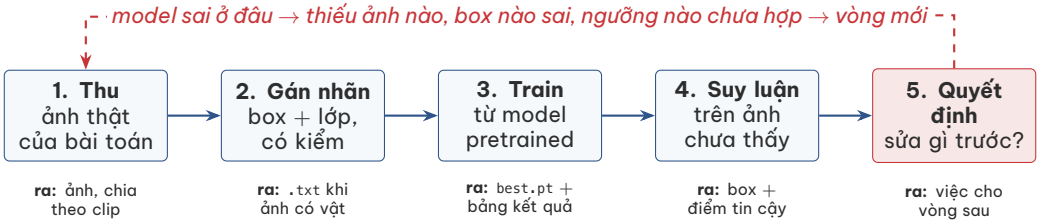
Bốn câu cần hỏi trước khi tin, và một việc cần làm trước khi dùng?

06

Xây Một Detector Từ Con Số 0

Dữ liệu tự làm từ số 0, model thì mượn: năm trạm đến một quyết định

Vòng lặp, không phải đường thẳng



Đầu ra của trạm này là đầu vào của trạm sau. Mục tiêu chiều nay **không phải** một model thật giỏi — bốn giờ không đủ cho việc đó — mà là **chạy trọn một vòng** và đọc được chỗ model sai.

Trước trạm 1: viết bài toán thành ba câu

Câu hỏi	Ví dụ cho lab chiều nay
Model dùng để làm gì, ai dùng kết quả?	cảnh báo trên màn hình khi có người đi bộ hoặc xe đạp phía trước camera
Lớp nào? Lỗi nào đắt hơn?	`pedestrian`, `cyclist`, `car`, `truck`; bỏ sót người đi bộ đắt hơn báo nhầm
Thế nào là đạt, đo trên ảnh nào?	recall của `pedestrian` trên ảnh giữ lại, chụp ở chỗ khác và giờ khác với ảnh train

Ba câu này quyết định thu ảnh nào, vẽ box cho vật nào, và cuối ngày nhìn con số nào. Đổi bài toán giữa chừng là làm lại từ trạm 1.

Trạm 1 — thu ảnh: bao nhiêu là đủ?

Con số tham khảo của Ultralytics

- Dùng thật: $\geq$**1.500 ảnh** và $\geq$**10.000 box** *mỗi lớp*.
- Thử cho biết: **vài trăm box mỗi lớp** đủ để thấy train tiếp từ model có sẵn hoạt động.
- Thêm **0–10% ảnh không có vật nào** để bớt báo nhầm (COCO: khoảng 1%).

Lab chiều nay

- Mỗi người khoảng 30 ảnh: **gộp cả lớp** để thử một vòng train chung, theo **một** quy tắc box.
- **Chia theo nơi chụp, theo clip** trước, rồi mới chia train/val (Ngày 1, Bước 5).
- Chụp theo quy tắc Ngày 1: **không lộ mặt, không lộ biển số** — đó là dữ liệu cá nhân.

Hai công cụ, hai vai trò

CVAT

Nơi dữ liệu được *tạo* và *duyệt*

- web, nhiều người cùng làm, có bước duyệt
- label schema, phím tắt
- AI vẽ nháp: SAM, detector có sẵn
  bản tự cài phải triển khai thêm Nuclio
- export Ultralytics YOLO, COCO

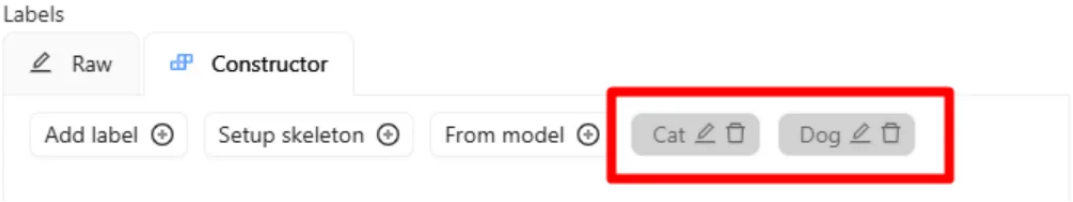
ultralytics

Nơi model được *train* và *đo*

- thư viện Python + lệnh `yolo`
- train, val, predict, export
- in precision, recall, mAP
- `auto_annotate`: vẽ nháp bằng model có sẵn

Nháp của máy chỉ gồm các lớp model đã biết, ra polygon, và chưa ai kiểm.

Label schema — chốt trước khi tạo dữ liệu

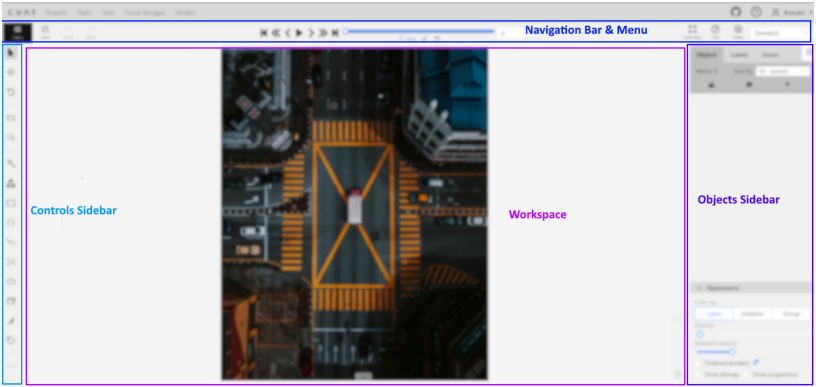


1. *Add label* — thêm một lớp vào danh sách
2. Mỗi lớp một **màu**, để nhìn ra ngay trên ảnh
3. *Raw* — dán cả schema dạng JSON

Khung đỏ = danh sách lớp đã tạo. Ví dụ của CVAT là `Cat`/`Dog`; lớp của mình sẽ là `pedestrian`, `cyclist`, `car`, `truck`.

Nguồn: docs.cvat.ai — MIT

Bốn vùng của CVAT editor



1. *Navigation bar* — chuyển frame, lưu, đổi chế độ **2.** *Controls sidebar* — dải icon dọc ngoài cùng bên trái
3. *Workspace* — ảnh đang mở, nơi box được vẽ và chỉnh **4.** *Objects sidebar* — danh sách vật thể đã gán, lớp và attribute

Nguồn: docs.cvat.ai — MIT

Trạm 2 — gán nhãn trong CVAT

1. **Tạo project** + khai **label schema**: lớp, màu, attribute.
2. **Tạo task**, upload ảnh. Muốn chia job ~100–200 ảnh thì đặt *Segment size* — mặc định là **một job cho cả task**.
3. **Vẽ box**: bấm công cụ *Rectangle*, chọn lớp, kéo box. Phím `N` lặp lại lần vẽ vừa rồi.
4. **Tự kiểm**, rồi **Export** → *Ultralytics YOLO Detection 1.0* (không phải *YOLO 1.1* — bố cục cũ). File YOLO chỉ giữ lớp + box: thuộc tính như `occluded` không đi theo — giữ thêm bản export *CVAT for images 1.1*.

Tốc độ đến từ bàn phím

`N` vẽ tiếp, `F` / `D` ảnh sau / trước, `Ctrl+S` lưu. Mọi phím tắt đổi được trong phần cài đặt.

YOLO format: một file `.txt` cho một ảnh

```
dataset/
images/train/frame_0001.jpg    # image
labels/train/frame_0001.txt    # boxes: SAME name, .txt, images/ -> labels/

# frame_0001.txt – one line per object
# <class_id> <x_center> <y_center> <width> <height>
2 0.4820 0.6135 0.0531 0.2139
0 0.1204 0.5570 0.0890 0.0745
```

Bốn số chia theo cỡ ảnh

Tâm *x*, tâm *y*, rộng, cao — chia cho cỡ ảnh nên nằm trong $[0, 1]$. Số đầu dòng = thứ tự lớp trong `names` của `data.yaml`.

Ảnh không có vật nào

Không cần file `.txt`; file rỗng cũng được, train như nhau. Nhưng ảnh *có* vật mà thiếu box là **dữ liệu sai**.

Cùng một box: YOLO `.txt` và COCO `.json`

YOLO — một file cho mỗi ảnh

```
2 0.5 0.5 0.1 0.2
```

- tâm + cỡ, chia theo cỡ ảnh
- cỡ ảnh **không** nằm trong file
- lớp là số thứ tự trong `data.yaml`: `car` = 2

COCO — một file cho cả tập (trích đoạn)

```
"images": [{"id": 7,
"width": 1920, "height": 1080}],
"annotations": [{"image_id": 7,
"category_id": 3,
"bbox": [864, 432, 192, 216]}]
```

- góc trên-trái + cỡ, tính bằng **pixel**
- lớp tra trong `categories`: COCO gốc ghi `car` = 3

Lưu ý: Đổi qua lại phải biết **cỡ ảnh**, **gốc toạ độ** và **bảng tra lớp**. Sai một trong ba thì box lệch hoặc sai lớp — và không có thông báo lỗi nào.

Từ export CVAT đến dữ liệu train và val

Export: `Ultralytics YOLO Detection 1.0`.

```yaml
# data.yaml
path: /full/path/to/dataset
train: autosplit_train.txt
val:   autosplit_val.txt
names: {0: pedestrian, 1: cyclist,
2: car, 3: truck}
```

Export từ CVAT chưa train được ngay

`data.yaml` của CVAT chỉ có `train:` — thiếu `val:` là lệnh train báo lỗi. Ảnh chụp rời: hai dòng Python dưới đây. Ảnh cắt từ video: chia theo clip bằng tay.

```python
from ultralytics.data.split import autosplit
autosplit("dataset/images", weights=(0.9, 0.1, 0.0))
```

`path:` tương đối tính từ **thư mục đang đứng** (không thấy thì từ thư mục `datasets` của Ultralytics) — ghi đường dẫn tuyệt đối.

Trạm 3 — train: ba lệnh, không cần viết code

Dùng dữ liệu đã chuẩn bị ở trang trước.

Train → chấm model → chạy trên ảnh giữ lại.

```
yolo detect train   data=data.yaml model=yolo11n.pt epochs=50 imgsz=640

yolo detect val     data=data.yaml model=runs/detect/train/weights/best.pt

yolo detect predict model=runs/detect/train/weights/best.pt source=holdout/
```

Đường dẫn ví dụ là lần chạy đầu. Lần train sau ra thư mục mới (bản 8.4: `train-2/`): dùng thư mục ở dòng `Results saved to`.

Trong 50 epoch, lệnh train làm gì?



- **Model mượn, dữ liệu tự làm**: `yolo11n.pt` đã học trên ảnh COCO; train chỉ chỉnh tiếp cho 4 lớp của lab (Ngày 1: hiếm khi train từ số 0).
- **Mosaic**: ghép 4 ảnh thành một để model gặp nhiều cỡ vật, nhiều bối cảnh; tắt ở 10 epoch cuối để model quen lại ảnh không ghép.
- **Hết mỗi epoch, chấm trên tập val**; giữ `last.pt` (epoch cuối) và `best.pt` (điểm val tốt nhất; bản 8.4 chấm theo mAP50-95) — dùng để suy luận.
- **Không dừng sớm**: `patience` mặc định 100 — phải 100 epoch liền không tiến bộ mới dừng, nên với 50 epoch thì không kích hoạt.

Đọc kết quả train: bốn mục cần xem

- `results.csv` — mỗi epoch một dòng. Dòng cuối là epoch *cuối*, chưa chắc là tốt nhất.
- `val_batch0_labels.jpg` cạnh `val_batch0_pred.jpg` — box có nhãn và box model đoán trên cùng ảnh: **xem bằng mắt**.
- `confusion_matrix.png` — lớp nào bị đọc thành lớp nào.
- `BoxPR_curve.png` — precision và recall của từng lớp khi hạ dần ngưỡng điểm.

Recall một lớp thấp — chưa vội kết luận

Có thể do vật nhỏ, cảnh đông (NMS xoá nhầm), hoặc box bị bỏ sót trong dữ liệu *train*. Còn nếu tập *val* thiếu box, dự đoán đúng bị tính là **báo nhầm**: precision tụt, không phải recall.

Confusion matrix: lớp nào bị đọc thành lớp nào

pred ↓ / *true* →	pedestrian	cyclist	car	truck
pedestrian	**0.92**	0.06	0.00	0.00
cyclist	0.05	**0.85**	0.01	0.00
car	0.01	0.05	**0.94**	**0.33**
truck	0.00	0.02	0.04	**0.65**

Trong ma trận minh hoạ, **33%** xe tải thật bị đọc thành `car`. Cần mở các ảnh bị nhầm. Box `truck` và `car` trong dữ liệu train cũng lẫn nhau ⇒ sửa **guideline**; không lẫn ⇒ có thể thiếu ảnh xe tải.

Cột = lớp thật, hàng = lớp model đoán. Ultralytics vẽ thêm hàng và cột *background* (bỏ sót, báo nhầm), ở ngưỡng điểm 0,25 và IoU 0,45 — khác chỗ tính mAP.

Trạm 4 — chạy model trên ảnh nó chưa từng thấy

```
yolo detect predict \
model=runs/detect/train/weights/best.pt \
source=holdout/ conf=0.25 save=True

# webcam instead of a folder: source=0 show=True
```

Ảnh có box vẽ sẵn nằm trong `runs/detect/predict/`.

Ảnh giữ lại là gì

Ảnh **không** nằm trong train hay val — tốt nhất chụp ở chỗ khác, giờ khác. Model đoán giỏi trên ảnh đã học chưa nói lên điều gì.

Đúng	**Bỏ sót**	**Báo nhầm**	**Lệch**
box khớp vật	vật có, không box	box, không có vật	đúng vật, box lệch

Mỗi lỗi ghi thêm **một chữ vì sao**: nhỏ, bị che, tối, góc lạ, hay vật chưa có trong schema. Danh sách này là đầu vào của trạm 5.

Sáu kiểu sai của detector (TIDE)

Lỗi	Nghĩa là	Gợi ý kiểm tra trước
Cls	đúng chỗ (IoU $\geq 0{,}5$), sai lớp	định nghĩa lớp; ảnh của các lớp dễ lẫn
Loc	đúng lớp, box lệch ($0{,}1 \leq$ IoU $< 0{,}5$)	quy tắc box sát; vật nhỏ
Both	sai lớp *và* lệch ($0{,}1 \leq$ IoU $< 0{,}5$)	thường là ca mơ hồ
Dupe	hai box cho một vật	NMS, ngưỡng IoU của NMS
Bkgnd	báo có ở chỗ không có box nào (IoU $< 0{,}1$)	vật thật mà **thiếu box**? nền trông giống vật?
Missed	vật có box, không dự đoán nào khớp	vật nhỏ, bị che, lớp ít ảnh

Trong 100 lỗi "Bkgnd" tự tin nhất của Mask R-CNN trên COCO, TIDE thấy 57 lỗi truy được về chính dữ liệu có nhãn: cờ crowd, box sai, hoặc lớp mơ hồ. (Bolya et al., ECCV 2020, Bảng 5)

Trạm 5 — sửa dữ liệu, đổi ngưỡng, hay đổi model?

Thấy gì trên ảnh giữ lại	Thử gì trước
Bỏ sót một loại vật ở *hầu hết* ảnh	dữ liệu train: vật đó có bị thiếu box không, có đủ ảnh không
Báo nhầm nhiều trên nền trông giống vật	thêm ảnh nền (0–10% tập train); kiểm box bị thiếu
Hai lớp lẫn nhau	đọc lại định nghĩa lớp; xem box của hai lớp trong dữ liệu train
Chỉ sai ở ảnh tối, mưa, góc lạ	thu thêm *đúng loại* ảnh đó, không phải ảnh bất kỳ
Đúng vật nhưng box lệch, lỏng	quy tắc box sát; xem mAP50-95, không chỉ mAP50
Đúng cả, chỉ ngưỡng chưa hợp việc	đổi `conf` — không cần train lại

*Mỗi vòng đổi **một** thứ rồi so trên **cùng** ảnh giữ lại; chỉ đổi ngưỡng thì không train lại. Thường kiểm dữ liệu trước; đã sạch mà vẫn yếu hay chậm: thử model lớn hơn hoặc kiểu khác.*

Ra khỏi phòng lab: export, chạy, theo dõi

```
yolo export model=runs/detect/train/weights/best.pt format=onnx      # CPU, many runtimes
yolo export model=runs/detect/train/weights/best.pt format=engine    # NVIDIA TensorRT
yolo export model=runs/detect/train/weights/best.pt format=openvino  # Intel CPU / GPU
```

Chạy trên máy nhỏ

NVIDIA Jetson Orin Nano Super: YOLO26n khoảng **4,6 ms mỗi ảnh** sau khi export TensorRT (số của Ultralytics, đổi theo phiên bản).

Điểm không phải xác suất

Box điểm 0,9 không có nghĩa đúng 90%. Một nghiên cứu đo điểm lệch trung bình **19 điểm phần trăm** so với tỉ lệ đúng thật (Faster R-CNN).

Theo dõi sau khi chạy

Lưu ảnh model **không chắc** (điểm gần ngưỡng) và ca người dùng báo sai — đó là ảnh cho vòng thu tiếp theo.

Thực Hành

07

Năm trạm trong bốn giờ: từ ảnh chụp đến một quyết định

Mục tiêu: Cả lớp gộp ảnh đường phố, tạo box theo schema bốn lớp và chấm với gold set, train một detector bằng Ultralytics, chạy nó trên ảnh giữ lại, rồi quyết định sửa gì cho vòng sau

Deliverable: Bảng lỗi trên ảnh giữ lại (bỏ sót / báo nhầm / lệch, kèm vì sao) + IoU của 30 box với gold set + giải thích bảng sát / $+10\%$ / $+25\%$ + **một** đề xuất cho vòng sau

Thời gian: 4 giờ

Năm trạm trong bốn giờ

Giờ	Trạm	Làm gì	Ra
13:30	1. Thu	gộp ảnh cả lớp; chia theo nơi chụp; giữ riêng ảnh để thử	thư mục ảnh, danh sách giữ lại
14:00	2. Gán nhãn	tạo box trong CVAT; 30 box chấm với gold set	export YOLO, IoU từng box
15:00	3. Train	một lần train chung; cả lớp đọc cùng một bảng kết quả	`best.pt`, `results.csv`
15:45	4. Suy luận	chạy model trên ảnh giữ lại; ghi lỗi và vì sao	bảng lỗi
16:30	5. Quyết định	tính IoU 0,83 bằng tay; đọc bảng sát / $+10\%$ / $+25\%$; đổi `conf` từ 0,1 đến 0,9	một đề xuất cho vòng sau

Bảng sát / $+10\%$ / $+25\%$ được tính sẵn trên hơn 500 ảnh, mỗi kiểu chạy 3 lần: tập ảnh của một buổi chiều quá nhỏ để thấy khác biệt thật giữa ba kiểu.

Năm điều hay hiểu nhầm

Hay nghĩ	Thật ra
Điểm 0,91 nghĩa là đúng 91%	điểm nội bộ của model; hai model cùng in 0,9 có thể đúng với tỉ lệ rất khác nhau
Vẽ box rộng ra một chút cho chắc	box lỏng 10% vẫn qua IoU 0,5, nhưng cho model một đích học to hơn vật thật
Trông khác nhau thì nên tách lớp	chỉ tách khi phía sau xử lý khác nhau *và* nhìn một ảnh phân biệt chắc
Bốn số của box ở file nào cũng cùng nghĩa	YOLO: tâm, chia theo cỡ ảnh; COCO: góc trên-trái, pixel; model in ra: hai góc
mAP cao nghĩa là dữ liệu tốt	mAP50 ít nhạy với box lỏng vừa phải; và mAP đo model, không đo người

Trước khi rời lớp: năm câu tự kiểm

- ☑ Nói được detector trả về gì, và vì sao điểm tin cậy không phải xác suất
- ☑ Tính được IoU bằng tay, và nói box đó qua những ngưỡng nào
- ☑ Giải thích được vì sao đổi ngưỡng ảnh hưởng đến precision và recall
- ☑ Chỉ ra một lỗi của model trên ảnh giữ lại, và đoán nó đến từ dữ liệu hay từ ngưỡng
- ☑ Nêu được một thay đổi cho vòng sau — và vì sao mỗi vòng chỉ đổi một thứ

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. Detection = lớp + điểm tin cậy + box. Dữ liệu train có lớp và box, chỉ thiếu điểm; bốn số ghi theo từng định dạng.
2. Vật bị bỏ sót trong dữ liệu train dạy model rằng ở đó không có gì — đó là dữ liệu sai, không chỉ thiếu.
3. IoU đo hai box trùng nhau bao nhiêu — dùng được cho cả model với người và người với người.
4. mAP50 ít nhạy với box lỏng vừa phải. Xem thêm mAP50-95, rồi mở ảnh ra xem model sai ở đâu.
5. Xây detector là một vòng: thu, gán nhãn, train, suy luận, quyết định — mỗi vòng đổi một thứ.

Tiếp theo & Bài tập

Ngày 03 — Multi-Object Tracking

*"Một box đúng trên một frame là chưa đủ: cùng một chiếc xe phải giữ **cùng một ID** qua nhiều frame, kể cả khi bị che — bài học của Tempe 2018. Ví dụ: 300 frame, bị che hai giây."*

- Đọc trước: Ultralytics Docs, mục *Track*
- Chuẩn bị: một clip 10–20 giây có ít nhất 3 vật thể di chuyển
- Mang theo: bảng lỗi và đề xuất của trạm 5

Tài liệu tham khảo (1/5): công cụ và cách đo

1. Ultralytics, *YOLO Docs — Detect, Train, Val, Predict, Export*. `https://docs.ultralytics.com`
2. Ultralytics, *Tips for Best Training Results*; *Data Collection and Annotation*.
3. CVAT, *Annotation with rectangles & Export formats*. `https://docs.cvat.ai`
4. COCO, *Detection Evaluation Metrics*. `https://cocodataset.org/#detection-eval`
5. Padilla et al., *A Comparative Analysis of Object Detection Metrics*, Electronics 10(3), 2021.
6. Everingham et al., *The PASCAL Visual Object Classes Challenge*, IJCV, 2010.
7. Bolya et al., *TIDE: A General Toolbox for Identifying Object Detection Errors*, ECCV 2020.
8. Küppers et al., *Multivariate Confidence Calibration for Object Detection*, CVPR Workshops 2020.

Tài liệu tham khảo (2/5): detector và công cụ 2026

1. Bochkovskiy et al., *YOLOv4*, arXiv:2004.10934, 2020.
2. Carion et al., *End-to-End Object Detection with Transformers (DETR)*, ECCV 2020.
3. Girshick et al., *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation* (R-CNN), arXiv:1311.2524 (CVPR 2014).
4. Ren et al., *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*, arXiv:1506.01497 (NIPS 2015).
5. Ultralytics, *YOLO11*, *YOLO26* model pages. `docs.ultralytics.com/models`
6. RT-DETR / RT-DETRv2 (`github.com/lyuwenyu/RT-DETR`); D-FINE, arXiv:2410.13842; RF-DETR (`github.com/roboflow/rf-detr`).
7. Liu et al., *Grounding DINO*, arXiv:2303.05499; YOLO-World, arXiv:2401.17270.
8. Meta, *SAM 3* (README, 11/2025). `github.com/facebookresearch/sam3`
9. *Roboflow100-VL*, arXiv:2505.20612; *OpenAD*, arXiv:2411.17761.
10. Shao et al., *CrowdHuman*, arXiv:1805.00123, 2018.

Tài liệu tham khảo (3/5): chất lượng dữ liệu có nhãn

1. Papadopoulos et al., *Extreme Clicking for Efficient Object Annotation*, ICCV 2017.
2. Papadopoulos et al., *We Don't Need No Bounding-Boxes*, CVPR 2016.
3. Su, Deng, Fei-Fei, *Crowdsourcing Annotations for Visual Object Detection*, AAAI HCOMP Workshop 2012.
4. Fort & Sagot, *Influence of Pre-annotation on POS-tagged Corpus Development*, LAW IV, 2010.
5. Wu et al., *Soft Sampling for Robust Object Detection*, arXiv:1806.06986.
6. Zimmermann et al., *Benchmarking a Benchmark: How Reliable is MS-COCO?*, arXiv:2311.02709.
7. Singh et al., *Benchmarking Object Detectors with COCO: A New Path Forward* (COCO-ReM), arXiv:2403.18819.
8. Tkachenko et al., *ObjectLab: Automated Diagnosis of Mislabeled Images in Object Detection Data*, arXiv:2309.00832.

Tài liệu tham khảo (4/5): đường phố, ứng dụng, sự cố

1. NTSB, *Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian*, HAR-19/03, 2019.
2. Yu et al., *BDD100K*, arXiv:1805.04687; Varma et al., *IDD*, WACV 2019; *DriveIndia*, arXiv:2507.19912.
3. ASEAN NCAP, *Assessment Protocol — Motorcyclist Safety*, v2.0, 05/2024.
4. Euro NCAP, *Test Protocol — AEB/LSS VRU Systems*, v4.4 (Implementation 2023).
5. U.S. FDA, De Novo DEN200055 (GI Genius); Google ML Kit, *Face Detection*.
6. *MegaDetector* (MIT). `github.com/agentmorris/MegaDetector`

Tài liệu tham khảo (5/5): họ YOLO

1. Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, arXiv:1506.02640 (CVPR 2016).
2. Redmon & Farhadi, *YOLO9000: Better, Faster, Stronger*, arXiv:1612.08242; *YOLOv3: An Incremental Improvement*, arXiv:1804.02767.
3. Long et al., *PP-YOLO*, arXiv:2007.12099 (YOLOv4: xem Tài liệu tham khảo 2/5).
4. Ge et al., *YOLOX: Exceeding YOLO Series in 2021*, arXiv:2107.08430; Li et al., *YOLOv6*, arXiv:2209.02976.
5. Wang et al., *YOLOv7*, arXiv:2207.02696; *YOLOv9*, arXiv:2402.13616.
6. Wang et al., *YOLOv10: Real-Time End-to-End Object Detection*, arXiv:2405.14458 (NeurIPS 2024).
7. Tian et al., *YOLOv12*, arXiv:2502.12524; Lei et al., *YOLOv13*, arXiv:2506.17733.
8. Jocher et al., *Ultralytics YOLO26*, arXiv:2606.03748; Ultralytics Docs, trang *YOLOv5*, *YOLOv8*, *YOLO11*, *YOLO12*, *YOLO26*; GitHub release *v8.4.142* (05/09/2026).
9. Feng et al., *TOOD*, arXiv:2108.07755; Li et al., *Generalized Focal Loss*, arXiv:2006.04388; Zheng et al., *Distance-IoU Loss*, arXiv:1911.08287.

Hỏi & Đáp



Cảm ơn!

Liên hệ: email@vinuni.edu.vn

Slide và lab: `Track4-DataAnnotation/Day2/`